■■ CyberBlackBox AI — Codebase Audit

Comprehensive Architectural Evaluation & Completion Plan • Forensic Standards Grade

Auditor: Senior Backend & AI Systems Auditor	**Target Branch:** main (commit 7bf718b)
Target Project: CyberBlackBox AI (Prototype)	**Working Tree:** Clean (0 local uncommitted diffs)
Audit Date: October 15, 2026	**Verification Mode:** Static Code Inspection & Programmatic TestClient

Executive Audit Summary: CyberBlackBox AI currently presents a **high-fidelity interactive UI prototype** with an operational FastAPI backend serving pre-correlated mock data. The application renders attack graphs, escalation curves, and court-ready PDF templates flawlessly. However, the **core dynamic forensics pipeline is not yet implemented**: no file upload endpoints exist, no multi-format parsers (TXT/CSV/JSON) are connected, event correlation is static, and RAG retrieval simulates vector similarity using string keyword matching rather than real embeddings. **Zero code modifications were made during this audit.**

A. Project Snapshot & Architecture

The project is structured as a decoupled two-tier client-server architecture: a FastAPI backend on port 8080 and a React 18 / Vite SPA on port 3000.

```
cyber-blackbox-prototype/
■■■ backend/ (main.py, models.py, requirements.txt, data/sample_incidents.py, data/mitre_kb.json, services/)
■■■ frontend/ (src/App.jsx, src/components/*, src/services/api.js, package.json, vite.config.js,
vercel.json)
■■■ root/ (jury_presentation_guide.txt, README.md, render.yaml, architecture_overview.md)
```

B. End-to-End Pipeline Feature Audit Table

Each forensic capability was inspected independently across source files and runtime behavior:

Capability	Status	Relevant Files & Endpoints	Audit Findings & Identified Deficiencies
1. Frontend Startup & Rendering	**WORKING**	frontend/src/App.jsx main.jsx, vite.config.js	Compiles cleanly with Vite (code 0). Renders all 3 prototype modes and tabs.
2. Evidence / File Upload	**NOT IMPLEMENTED**	frontend/src/ (no UI) backend/main.py	No exists in UI. No /upload route in backend.
3. Validation & Error Handling	**NOT IMPLEMENTED**	None	Zero MIME validation, file size enforcement, or payload sanitization.
4. Multi-format Log Parsing	**NOT IMPLEMENTED**	frontend TraditionalLogViewer sample_incidents.py	No parsing logic (regex, CSV, JSON). Raw logs stored as static strings.
5. Event Normalization	**PARTIALLY WORKING**	backend/models.py LogEntry schema	Standard schema exists in Pydantic, but populated solely via mock dictionary.
6. Evidence Integrity & SHA-256	**PARTIALLY WORKING**	backend/services/ hash_service.py	Computes deterministic SHA-256 over dictionaries. Missing raw file streaming hash.
7. Cross-Source Correlation	**NOT IMPLEMENTED**	backend/data/ sample_incidents.py	No temporal clustering or rule engine. Correlations are manually authored.
8. Chronological Timeline	**PARTIALLY WORKING**	frontend AttackTimeline.jsx backend/models.py	Recharts escalation curve renders cleanly. Backed by static scenario timeline.
9. Root Cause Identification	**NOT IMPLEMENTED**	DigitalForensicsDeepDive.jsx sample_incidents.py	Verdict (Spearphishing) and confidence (98.4%) are hardcoded string constants.

10. Affected Asset Discovery	**NOT IMPLEMENTED**	backend/data/ sample_incidents.py	Compromised assets array is hardcoded. No dynamic extraction from host logs.
11. Attack Graph Reconstruction	**PARTIALLY WORKING**	frontend/src/components/ AttackGraph.jsx	SVG/HTML cards render topology. Edge generation is not derived from logs.
12. MITRE ATT&CK; Mapping	**PARTIALLY WORKING**	backend/data/mitre_kb.json services/rag_engine.py	14 techniques loaded into memory. Missing automated signature classifier.
13. Dynamic Risk Scoring	**NOT IMPLEMENTED**	backend/data/ sample_incidents.py	Risk score 96/100 is static. No mathematical formula evaluates event severity.
14. Automated Containment Actions	**PARTIALLY WORKING**	backend/main.py (/execute) ResponsePlaybooks.jsx	Endpoints update in-memory state to EXECUTED. Playbooks are hardcoded.
15. Vector RAG Engine	**PARTIALLY WORKING**	backend/services/ rag_engine.py	Uses word-overlap formula instead of embeddings. Falls back to hardcoded if/elif text.
16. Forensic Report Export	**WORKING**	backend/services/report_service ForensicReportModal.jsx	Calculates master evidence digest, displays formatted report, prints to PDF.
17. Frontend-Backend API Link	**WORKING**	frontend/src/services/api.js backend/main.py	Endpoints tested with HTTP 200 OK. Resilient offline fallback data protects UI.
18. Persistence & Production	**BROKEN / INCOMPLETE**	render.yaml, vercel.json backend/main.py	Zero database persistence. Zero tests. Hardcoded localhost URLs.

C. End-to-End Data Flow Analysis

Where the user pipeline currently terminates: When a user accesses the platform with a raw forensic log file (e.g., Linux auth.log, Windows Event CSV, or AWS CloudTrail JSON), **the pipeline terminates at step 0**. The web interface does not contain an upload button, drop zone, or file ingestion handler.

Current Internal Execution Flow (Pre-Loaded Mock Pipeline):

1. **Mount & Query:** Browser loads App.jsx and calls api.getIncident('INC-2026-8891').
2. **Backend Response:** FastAPI deep-copies SCENARIOS['INC-2026-8891'] and executes HashService.process_log_entry().
3. **Dashboard Render:** React displays the pre-calculated attack graph, 5-step timeline, and 6 mock logs.
4. **RAG Assistant Query:** User submits question ➔ POST /api/incidents/.../rag ➔ executes RAGEngine.retrieve() (word-overlap search).
5. **Ollama Check & Fallback:** Tries localhost:11434 (2s timeout). When offline, falls back to hardcoded keyword checks (if 'entry' in query...).
6. **Containment & Export:** User triggers containment action (toggles in-memory state) or exports court-ready report with SHA-256 digest.

D. Empirical Verification & Test Results

All tests were run via programmatic CLI checks during the audit without modifying project code:

Verification Scope	Command Executed	Outcome / Exit Code	Auditor Finding
Repo Test Coverage	`Get-ChildItem *test*`	0 tests found	No automated test suite exists in backend or frontend.
Backend Module Import	`python -c 'import main'`	**Exit Code 0 (Success)**	FastAPI app instance initializes without syntax or import errors.
FastAPI TestClient	`TestClient(app).get(...)`	**HTTP 200 on all routes**	/health, /api/scenarios, /api/incidents, and /rag routes return 200 OK.
Frontend Build Check	`npm run build`	**Exit Code 0 (10.37s)**	Vite bundles 2,350 modules cleanly with zero JSX/CSS errors.

E. Remaining Work Ranked by Engineering Priority

Priority P0 (Core Pipeline Blockers):
• **Parser Service:** Build backend/services/parser_service.py to ingest TXT, CSV, and JSON logs.
• **Upload Endpoint:** Implement POST /api/evidence/upload using python-multipart.
• **Frontend Upload UI:** Create drag-and-drop file upload zone in React frontend.

Priority P1 (End-to-End Dynamic Correlation):
• **Correlation Engine:** Build correlation_engine.py to sort timestamps and link IP/process entities.
• **Dynamic Attack Graph & Timeline:** Derive nodes, edges, and timeline steps directly from parsed logs.
• **Automated Root Cause Verdict:** Identify initial access event heuristically.

Priority P2 (Forensic Intelligence & Reliability):
• **True Dense Embeddings:** Replace keyword overlap with sentence-transformers & FAISS vector index.
• **MITRE Signature Classifier:** Rule-based pattern matching against mitre_kb.json.
• **Automated Test Suite:** Add pytest unit tests for parsers, hashing determinism, and API validation.

Priority P3 (Persistence, Hardening & Deployment):
• **SQLite Session Store:** Persist user uploads and incident records.
• **Config & Deployment:** Replace hardcoded URLs with environment variables and finalize Render/Vercel live deployment.

F. Implementation Roadmap & First Task

Roadmap Milestones:
1. **Milestone 1 (P0):** Raw Ingestion & Multi-Format Parser Service (TXT, CSV, JSON).
2. **Milestone 2 (P1):** Dynamic Cross-Source Event Correlation, Attack Graph & Timeline Generator.
3. **Milestone 3 (P2):** Real Vector Embeddings (FAISS / MiniLM) & Automated MITRE Classification.
4. **Milestone 4 (P3):** SQLite Persistence, Automated Test Suite, and Cloud Production Deployment.

Recommended First Implementation Task:
Milestone 1, Task 1.1 — Implement parser_service.py & POST /api/evidence/upload.
Rationale: Ingestion is the fundamental prerequisite for every downstream capability. Adding this service introduces zero breaking changes to the existing working prototype, establishes immediate cryptographic evidence seals on upload, and is 100% testable via unit tests with sample log files.